
P3 - Rejection Sampling: A General Prompting Strategy to Encourage Exploration

Nishad Jahan.
Department of Computer Science
Aarhus University
au804630@uni.au.dk

Priyanka Anantha Narayanan
Department of Computer Science
Aarhus University
au777930@uni.au.dk

Thomas Lyhne Hansen
Department of Computer Science
Aarhus University au593483@uni.dk

Abstract

In this project, we demonstrate an alternative prompting strategy to the classic chain-of-thought prompting when dealing with fixed-answer math problems. The prompting strategy is a multi-agent/multi-role strategy for structuring how the model reasons and forces the model to explore a vast solution space. At its core, it's a solution sampling method. The main roles are *Solver* and the *Rejector*, where the Solver role must generate a set of unique answers. Then, the Rejector's job is to reject whatever the solver outputs. This is to counteract a common issue, where a model stops improving after the early suggestions due to overconfidence and poor self-evaluation. After rejection sampling, the model is asked to rank each of the proposed solutions. We demonstrate that using the proposed technique on our problem sample outperforms Chain-of-Thought prompting, and is on equal footing with Reflexion and Tree-of-Thought prompting on final question-answers from the MathArena dataset.

1 Introduction

Due to the increase in model reasoning capabilities, including mathematical reasoning, several independent benchmarks have been created to test these models in different domains. However, as a side effect of the data scaling laws, i.e. that the increase in model performance is proportional to the amount of data trained on, there is a risk of dataset contamination. By contamination, we mean that the model has been trained on the very benchmark it has been evaluated on. As such, the MathArena benchmark has been created by Balunovic et al. [2025b] in an attempt to give these models a fair comparison in mathematical reasoning, as well as keeping track of which models were released after the benchmark was created.

MathArena is a collection of recurring math competitions with high quality and great difficulty. These include competitions like AIME, HMMT and USAMO. In addition, for MathArena, 12 very hard final answer problems were handpicked in 2025, posing a great challenge to SOTA models. As of August 18 2025, the SOTA models only got about 5% Balunovic et al. [2025a].

In a blog post, Balunovic et al. [2025c], the authors behind MathArena discussed issues with LLM tool usage. Here they found that models often become overconfident and tends to get stuck / cease to explore. We believe this is not only related to tool usage, but also for LMM when doing mathematical reasoning (or any reasoning). Since the models in those settings often cannot get a response from an environment (unless the problem is program verifiable and the model has access to tools), several techniques have been proposed to extend the standard Chain-of-Thought (Wei et al. [2022], Kojima et al. [2022]) problem to structure the thinking process of the LLM. For example, Tree-of-Thought

(Yao et al. [2023]) and Reflexion (Shinn et al. [2023]). Reflexion-type methods use some kind of evaluator agent / self-assessment agent to act as the environment feedback to the model. While such methods have been shown to improve the model’s performance, some recent studies like Luo et al. [2025]¹ suggest that output confidence is not a reliable measure. We interpret this as: Models are not reliably good at evaluating themselves. We therefore propose a simple and concrete 2-phase prompt strategy to force the model to explore new solutions. The first phase is forced exploration or what we call *rejection sampling*: Two roles are played by the model (or even different models): 1) A Solver, which tries to solve the problem in CoT style (with a few more restrictions) and a Rejector, which rejects *any* and *all* inputs the solver proposes. The second phase is the ranking phase or extraction phase, where the model evaluates and ranks each of the proposed solutions. We demonstrate that in our setup, Rejection-Sampling performs at least as well as Tree-of-thought and Reflexion. Furthermore, even when not ranking the correct answer first, it often provides the correct solution somewhere in its rankings, which we believe is more useful than not having any correct answer at all.

2 Contributions

Our contributions are threefold: Firstly, we demonstrate that current models *are* capable of providing correct answers to questions that could not be solved previously using CoT in the MathArena dataset. Secondly, we demonstrate that the raw performance of a selected set of models is increased by applying our proposed method. Thirdly, the method we propose is general and simple. It should be easy to convert to any other domain without too much coding setup or hyperparameter selection (only one hyperparameter), since it can essentially be wrapped around the model. This is unlike Reflexion and Tree-of-Thought, which require task-specific engineering and decisions on many hyperparameters. Furthermore, the method is simple enough that it can easily be executed by a user in a regular chat interface (though without the input cost saving part, which we present later) by the user providing the Rejector a role without a single line of code.

3 Related Work

Several prompt engineering strategies exist and are in use today; however, we only discuss the most relevant ones for this paper / those used as baselines here.

Chain of thought prompting (CoT): Wei et al. [2022] demonstrated that breaking the prompt into smaller intermediate reasoning steps improves the model’s performance on tasks like arithmetic, common sense, and symbolic reasoning. Although originally proposed in Wei et al. [2022] as a few shot methods, which means CoT reasoning examples were provided, Kojima et al. [2022] showed that simply prefixing the prompt with “Let’s think step by step” or a zero shot chain of thought prompts also greatly improves model performance on several tasks (though still poorer performance than few shot CoT).

Tree of thought (ToT): Yao et al. [2023] proposes a programmatic extension of CoT, where thinking steps are structured as a DFS or BFS tree search, where each node is a thought step. It has 3 components: 1) A *thought generator*, which, given a node, generates k (hyperparameter) children / next candidate thought steps. Here, the paper proposes two variants: One *sample* method, where each sample is collected "independently" - meaning you ask the model n times, without revealing the other options. The other is the *propose prompt* method, where the model generates one time CoT to produce n samples. 2) *State evaluation*, which, given several sibling nodes (layer in the tree), assigns the plausibility of each node. Again, two variants are proposed: A *value* method, where the model is asked to give each node a value and a *vote* method, where the model is asked to vote on the nodes. 3) *Tree search* using DFS or BFS. The authors demonstrate improved performance over CoT on three chosen tasks: an arithmetic game called Game of 24, a 5x5 crossword puzzle and creative writing.

Reflexion (Rfx): Reflexion by Shinn et al. [2023] incorporates self-reflection through external feedback signals. The model tries to find a solution, gets feedback from evaluations, and reflects on its errors to improve future attempts. Unlike the usual Chain-of-Thought prompting, where reasoning is solely linear, Rfx establishes a feedback loop. This loop imitates reinforcement learning by storing reflective summaries in memory and guiding future reasoning based on these reflections. A key

¹The mentioned study Luo et al. [2025] is still a preprint

feature of Rfx is the separation between generation, evaluation, and reflection. The evaluator judges whether a proposed solution is correct or flawed. The reflector looks at the failure and suggests improvements without solving the problem directly.

4 Proposed Method

4.1 Basic form

Our method has 1 hyperparameter, the number of steps n , and two phases:

Phase 1 - Rejection sampling : Here, we let the chosen model act in two different roles.

One role is the Solver, see prompt in appendix 4. The solver is instructed to do the following: 1) Use CoT reasoning to derive an answer. 2) Don't be too confident. 3) Use the early parts of the text as thinking, not as a final paper submission. 4) Before the final answer, you must list all the previous answers and a short summary of their argument. Your new answer must be unique and not in this list.

The key part is the 4th instruction. We found that letting the model refresh its memory of this list right before it answers helps it create unique answers.

The other role is the Rejector, see prompt in appendix 5. It is instructed to do the following: 1) You know the answer, but won't share it. 2) Reject all the solver's answers and arguments. 3) Remind the solver to propose new unique answers and restate the list of previous answers. 4) Remind the solver not to guess. 5) Remind the solver to be explorative and use the knowledge built up so far.

The key step is the 2nd instruction. The rejector does not actually know the answer. However, it will act as if it knows the answer, and force the solver to produce something new.

The number of steps denotes how many times you want this dialogue to run.

Phase 2 - Ranking: Phase 1 only samples many candidate solutions. In real usage, you would like an actual answer. We propose a simple ranking prompt, see appendix 7, but any method for selection among multiple candidates could be used here. The benefit of ranking is that now we can compare models not only on the final answer / first ranking, but which rank they give the final answer, for instance, in the form a top 5 accuracy.

4.2 Summarized form

The issue with the basic form is, that the number of input tokens is quadratic in n . For very hard problems, you might want to set $n = 50$, yielding a huge cost for current SOTA models. We therefore propose a summarized variant of the basic form. The idea is to add a third role, a *Summarizer*, acting as the conversation history. Thus, the input tokens are kept linear in n .

Let $\mathcal{Q}$ denote the space of questions / problems to solve. Let $\mathcal{I} = \{\text{SOLVER}, \text{REJECTOR}, \text{SUMMARIZER}\}$ denote the set of role instructions. Let $\mathcal{S}$ denote the space of strings, and let $[s_1, s_2, \dots, s_n]$ denote concatenation of strings s_i for $i = 1, \dots, n$. Finally, let $P : \mathcal{Q} \times \mathcal{I} \times \mathcal{S} \rightarrow \mathcal{S}$ denote a prompt-to-model-output mapping. Given the number of steps n and a question $q \in \mathcal{Q}$, we have the following algorithm:

Algorithm 1 Summarized Rejection Sampling (n, q)

Require: Number of steps n , question q

- 1: Initialize $Z_{i-2} \leftarrow \emptyset, Z_{i-1} \leftarrow \emptyset, R_{i-1} \leftarrow \emptyset, S_{i-1} \leftarrow \emptyset$
- 2: **for** $i \leftarrow 0$ to $n - 1$ **do**
- 3: $S_i \leftarrow P(q, \text{SOLVER}, [Z_{i-2}, S_{i-1}, R_{i-1}])$
- 4: $R_i \leftarrow P(q, \text{REJECTER}, [Z_{i-1}, S_i,])$
- 5: $Z_i \leftarrow P(q, \text{SUMMARIZER}, [Z_{i-1}, S_i, R_i,])$
- 6: $S_{i-1} \leftarrow S_i$
- 7: $Z_{i-2} \leftarrow Z_{i-1}$
- 8: $Z_{i-1} \leftarrow Z_i$
- 9: $R_{i-1} \leftarrow R_i$
- 10: **end for**
- 11: **return** Z_{n-1}

One note on terminology: For the basic version, n refers to how many conversations are swapped between each role. This is because the basic framework is more general and could be extended with more roles. However, in the case of summarized version, the responses of the Solver, Rejector and Summarizer are tightly coupled. Therefore, from now on we refer to n swaps when talking about the basic version. For instance, $n_swaps = 100$ means 50 proposed solutions get generated. When talking about the summarized version, we refer to n steps, n iterations of the algorithm. For example, $n_steps = 100$ means 100 proposed solutions are generated.

5 Experimental setup

Problem selection We selected a subset of the questions and models. For the most effective demonstration, we enumerate each problem by the number of correct attempts in the MathArena benchmarking (4 per model). Then we divided the problems in the data set into 10% percentiles in terms of the number of correct attempts. We then mapped this to “levels of difficulty” (descending). So, a level 1 problem is the hardest, meaning it lies in the 10th percentile (few or no models got the question right), and a level 10 means it lies in the 90th percentile (most or all models got this question right). For each model in our experiment, we then selected only problems it never got right among any of the 4 attempts in the MathArena benchmark, and we selected one question for each level of difficulty.

Model selection: When choosing models, we tried to reach a balance between token cost and performance on MathArena. These were: Deepseek-R1, GPT5-nano, Gemini 2.5-Flash

5.1 Baselines

We work with 3 baselines:

CoT: This is the “weakest” baseline, which is already the approach used in the MathArena dataset. So here we just collect the attempts from MathArena.

Rfx: In our implementation, we use Reflexion as a baseline with distinct roles for an *Evaluator*, a *Reflector*, and a *Reflexion-aware solver*, closely following the structure of the original paper, pulled from the Official Reflexion Repository. There were a few modifications made to make it work for our use case. The evaluator gives binary correctness signals. The reflector identifies the main reasoning error and suggests alternative directions. This setup allows for improvement over multiple attempts, but unlike our proposed rejection-sampling approach, Reflexion does not specifically promote solution diversity. As a result, while Reflexion improves answer quality through refinement, it may still struggle with limited exploration when the initial reasoning paths are systematically biased or incorrect.

ToT: Tot is very task-specific in that you need to code your task-specific prompts. For this, we pulled the original repository from Official Repo of Tree of Thoughts from Princeton and modified it for our use case. Furthermore, the method has a hyperparameter, which is the number of reasoning steps (tree depths), as well as which thought generator and state evaluation method to use. We used sampling as a thought generator and voting as a state evaluation. For the number of reasoning steps, we increased the number of reasoning steps as the task difficulty score went up (or if no model solved the problem).

A very important point of the benchmarking is, that the parameters were chosen in an ad hoc fashion. It very much possible, that changing the parameters of Rfx and ToT or do a different encoding of the tasks could result in better performance. Ideally, we should have run some kind of hyperparameter search with the different methods, and run multiple attempts for each model. However, due to the limits of scope and budget of this project, this was not done.

6 Results

Gemini.2.5 flash: See figure 1. As seen, the summarized method includes the correct answer in its proposals in all attempts, but does not always rank the correct answer first. For this particular session, we only run the summarized version for the level 1 difficulty problem, since the token cost would be very large for the basic version, and Gemini-2.5 flash is one of the more expensive models.

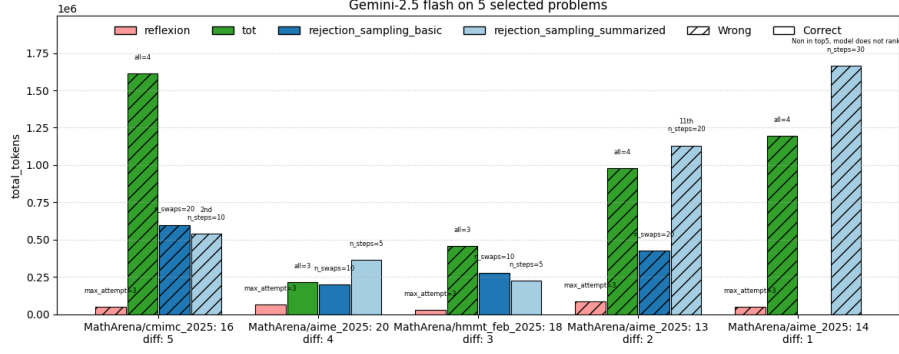


Figure 1: Performance of Gemini 2.5 Flash. The color indicates which method is being used. The dashed lines denote whether it has answered correctly. In the case of our method, we only deem it correct if it ranked the right answer first. The text above the text denotes the hyperparameters and (in the case of our methods) the ranking, if the model provided the right answer, but did not rank it first.

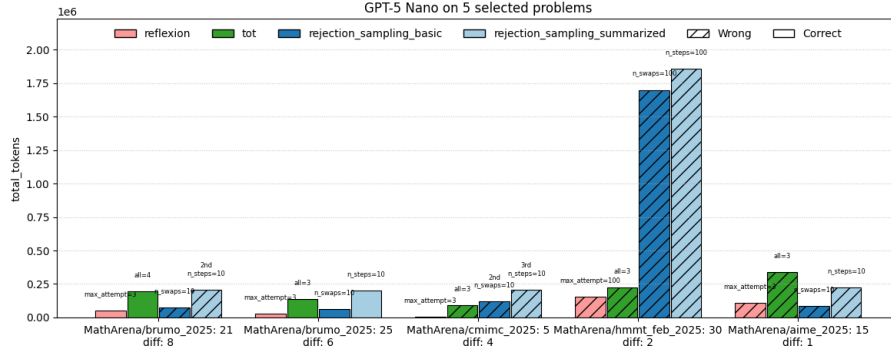


Figure 2: Performance of GPT5-Nano

GPT5-Nano: See figure 2. The model seemed to be unable to solve the last two difficulty levels in all methods. So we chose to increase the number of proposed solutions greatly on the second difficulty level, but still, no right solution was even among the rankings. Similarly, we increased *max_attempts* of reflexion and our methods to 100. In hindsight, the ToT hyperparameters should also have been increased. Notice that here the summarized version provides twice as many solutions as the basic version, while having almost the same cost.

DeepSeek-R1: See figure 3. Here we did not run the summarized version, as it was flagged by Azure as jailbreaking. The Rejection-Sampling method ranked the answer for all levels except for the Difficulty Level 2. For the last level, we decided to increase the number of steps to 60 for Reflexion and Rejection-Sampling, and they both arrived at the correct conclusion for the question.

7 Discussion

As can be seen from the figures, in our runs, our method consistently perform as well as baselines. Additionally, in many cases it is able to include the right answer, though it might not rank the correct answer first. The weakest link is definitely the ranking, as here, the model needs now evaluate it self.

The method is token-heavy, at least compared to reflexion. For all methods, there is a lot of variability in terms of token cost per message. In terms of tokens, it’s somewhat comparable to the tree-of-thought. However, that is heavily dependent on the hyperparameters of both methods. Nevertheless, it’s usually much more expensive than Rfx, since Rfx is able to exit the conversation as soon as it thinks it gets the right answer. This is cost-effective, but can also result in the model wrongfully accepting a wrong answer. See A.5

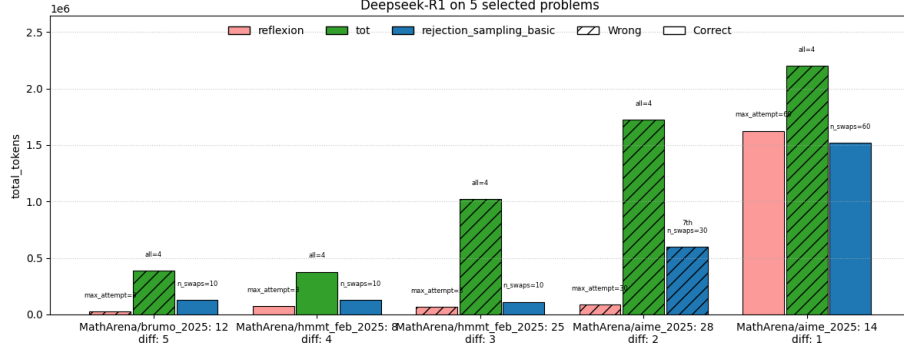


Figure 3: Performance of Deepseek-R1.

8 Conclusion

In this paper we demonstrated that models are able to solve some of the harder final answer problems that they previously could not solve in the MathArena dataset. We have provided a general single hyperparameter method (along with more cost effective method) that performs on par with ToT and Rfx, with the additional benefit that it often provides the correct answer somewhere in its ranking.

9 Future work

Final Answer problems: Due to the scope of the project, only final answer problems were selected. As the MathArena paper, Balunovic et al. [2025b], states, this does not adequately demonstrate mathematical multistep reasoning. An extension to this paper should therefore evaluate the same approach on proof-based questions.

Hard problem Selection: While our paper demonstrates that models can give the answer to some of the problems it did not get right even once in the original benchmarking, it does not explore if the technique deteriorates performance on questions the model could answer using CoT only. It’s possible that the increased number of input tokens might confuse the models.

Repeated runs: Another scoping issue of this paper is, the number of runs on each problem sample. Models answers can vary; therefore, for MathArena’s own benchmarks, Balunovic et al. [2025b], the authors run each model 4 times per question.

Ad hoc hyperparameters: A far more effective demonstration of the proposed method would have been to run all the problems with a fixed n , and then do this for several values of n . Such a study would also reveal more about the impact of n . Similarly, some kind of structured strategy for the baseline hyperparameters or a hyperparameter search would be useful. In our experiments, the hyperparameters of our method where increased with the problem difficulty level, but not consistently for the baselines. This creates possible an unfair advantage.

Token saving: A possible extension to the methods is to use a cheap model to do the rejection sampling, but then have a smarter, more expensive model to do the final ranking. As we have observed, though the model is able to generate the correct answer in instances where reflection, ToT, and CoT could not, it sometimes does not rank the correct solution as the most promising. Rather, the real value is somewhere in the rankings.

Combination with other methods: Since our idea is a sampling method, it could be used in the sampling phase of other methods, such as Tree-of-thought. Thus it could leverage both the sampling power of our method, as well as the iterative improvement nature of such methods.

References

Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena apex: Unconquered final-answer problems, 2025a. URL <https://matharena.ai/apex/>. Blog post, accessed 16 Dec 2025.

Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating LLMs on uncontaminated math competitions. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025b. URL <https://openreview.net/forum?id=y0zL9IZxZ7>.

Mislav Balunovic, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Agentic euler: Which project euler problems are within reach of llms?, 2025c. URL <https://matharena.ai/euler/>. Blog post, accessed 16 Dec 2025.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Jinyuan Luo, Zhen Fang, Yixuan Li, Seongheon Park, and Ling Chen. Shaking to reveal: Perturbation-based detection of llm hallucinations, 2025. URL <https://arxiv.org/abs/2506.02696>.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=vAE1hFcKW6>.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=5Xc1ecx01h>.

A Prompt templates

A.1 Solver Prompt

```
"""
You solve problems.
You try to reason step by step. You are not too confident in your answers (in the
sense you are open to be wrong), but rather you rely on fully fleshed out
mathematical reasoning.
You try to explore many ideas. Everytime you speak you will propose a fresh answer.
You dont submit the same answer twice. Everytime you come with a new answer, you
state all the previous answers in a list in format of tuples: (Answer, short
summary).
For example,
[
(780, induction on N, and lower bound on Z/N),
(28/2, CLT of H and proof by contradiction of Z>N)
]
Then you check that your new proposal is not in that list. If it is, you try again.
Use the early parts of your prompt as thinking text, not "for science paper style"
- meaning you can write your things and doubts.
Ex "I am thinking there could be a hint in the upper bound. I will check it out.
Ahh, I see I made a mistake. But now the size formula seems really promising!"
"Then formalize and submit answer when you think you have something. While you know
the rejector is skeptical, you really try to find the right answer every try.
"""
```

Figure 4: Solve prompt

A.2 Rejector prompt

```
""
You reject everything the Solver tries. You tell him his answer is wrong and his
reasoning is wrong.
You know the the right answer, but will not share it. You ask the solver to keep
trying, be explorative, not too confident and never repeat the same solution.
Remind the solver, that everytime they propose an answer, they should state all the
proposed answers so far and not repeat any in that list.
Ask the Solver not to be biased by the previous outcomes. Remind the solver not to
guess, but to use reasoning and the accumulated knowledge so far. Keep it
brief (rejector - Solver has no limit to his reason arguments)
""
```

Figure 5: Rejector prompt

A.3 Summarizer prompt

```
""
You summarize the conversions so far. You try to extract the core arguments, and
results by the solver and what has been rejected by the rejecter.
You give this summary as context to the solver and rejector, so they ca continue
from there.
Most importantly, you provide at the very end of your message the list of current
answers and their short summaries. This is the most important
part, so that the solver can recreate past rejected solutions. Try to keep
everything breif, but informative.
""
```

Figure 6: Summarizer prompt

A.4 Ranking prompt

```
""
You have gone through several different choices for an answer. While the rejector
rejected all of the answers, one of them is actually correct.
Your job is to go through all of these answers using the knowledge you have
acquired, try to argue how each of the answers could be true,
then rate and rank these answers. I want to see a list of all answers ranked on
plausibility.
Rules:
- Do not submit a new answer. Just rate existing ones.
""
```

Figure 7: Ranking prompt

A.5 Reflexion output - failure

An example of the evaluator agent incorrectly accepting that the answer is correct and terminating the session - Difficulty level-2 for the problem: 'MathArena/aime_2025: 14'. The correct answer is 60.

```
""
=== Attempt 1 ===
```

Given a convex pentagon $\backslash(ABCDE\backslash)$ with $\backslash(AB = 14\backslash)$, $\backslash(BC = 7\backslash)$, $\backslash(CD = 24\backslash)$, $\backslash(DE = 13\backslash)$, $\backslash(EA = 26\backslash)$, and $\backslash(\angle B = \angle E = 60^\circ\backslash)$, we need to find the least possible value of $\backslash(f(X) = AX + BX + CX + DX + EX\backslash)$ for any point $\backslash(X\backslash)$ in the plane. The answer is expressed as $\backslash(m + n\sqrt{p}\backslash)$, where $\backslash(m\backslash)$ and $\backslash(n\backslash)$ are positive integers and $\backslash(p\backslash)$ is not divisible by the square of any prime. We need to find $\backslash(m + n + p\backslash)$.

1. ****Coordinates Assignment**:**

- Place $\backslash(B\backslash)$ at $\backslash((0, 0)\backslash)$ and $\backslash(A\backslash)$ at $\backslash((14, 0)\backslash)$.
- Calculate coordinates of $\backslash(C\backslash)$ using $\backslash(BC = 7\backslash)$ and $\backslash(\angle B = 60^\circ\backslash)$:

$$\backslash[$$

$$C = \backslashleft(3.5, \frac{7\sqrt{3}}{2}\backslashright)$$

$$\backslash]$$
- Determine coordinates of $\backslash(D\backslash)$ and $\backslash(E\backslash)$ using the given side lengths and angles. Through complex calculations involving reflections and rotations, we find:
 - $\backslash(D = \backslashleft(\frac{313}{14}, \frac{169\sqrt{3}}{14}\backslashright)\backslash)$
 - $\backslash(E = \backslashleft(\frac{72}{7}, \frac{104\sqrt{3}}{7}\backslashright)\backslash)$

2. ****Equilateral Triangles and Reflections**:**

- Construct equilateral triangles on $\backslash(AB\backslash)$ and $\backslash(EA\backslash)$:
 - For $\backslash(AB\backslash)$, the third vertex $\backslash(F\backslash)$ is $\backslash((7, 7\sqrt{3})\backslash)$.
 - For $\backslash(EA\backslash)$, the third vertex $\backslash(G\backslash)$ is $\backslash(\backslashleft(\frac{241}{7}, \frac{65\sqrt{3}}{7}\backslashright)\backslash)$.

3. ****Distance Calculation**:**

- Calculate the distance between the third vertices $\backslash(F\backslash)$ and $\backslash(G\backslash)$:

$$\backslash[$$

$$FG = 16\sqrt{3}$$

$$\backslash]$$

4. ****Sum of Distances**:**

- The minimal value of $\backslash(f(X)\backslash)$ is found by considering the sum of distances from a point that connects these equilateral triangles and the original vertices. The minimal sum is determined to be $\backslash(40 + 16\sqrt{3}\backslash)$.

5. ****Final Answer**:**

- The values $\backslash(m = 40\backslash)$, $\backslash(n = 16\backslash)$, and $\backslash(p = 3\backslash)$ give the result $\backslash(m + n + p = 40 + 16 + 3 = 59\backslash)$.

$$\backslash[$$

$$\backslashboxed{59}$$

$$\backslash]$$

Score: true

Feedback:

The solver's answer is correct. The key steps involve constructing equilateral triangles on sides $\backslash(AB\backslash)$ and $\backslash(EA\backslash)$, then calculating the distance between their third vertices $\backslash(F\backslash)$ and $\backslash(G\backslash)$. The minimal value of $\backslash(f(X)\backslash)$ is found by combining the lengths of $\backslash(AB\backslash)$, $\backslash(EA\backslash)$, and $\backslash(FG\backslash)$, leading to $\backslash(40 + 16\sqrt{3}\backslash)$. The final answer $\backslash(m + n + p = 59\backslash)$ is accurate.

****Self-reflection**:** The solver could improve by explicitly justifying why the minimal sum equals $\backslash(AB + EA + FG\backslash)$. Clarifying the geometric principle (e.g., using properties of Fermat-Torricelli points or transformations) would strengthen the reasoning. Additionally, verifying intermediate steps (e.g., coordinate calculations) ensures accuracy.

true

FINAL SOLUTION:

Given a convex pentagon $\backslash(ABCDE\backslash)$ with $\backslash(AB = 14\backslash)$, $\backslash(BC = 7\backslash)$, $\backslash(CD = 24\backslash)$, $\backslash(DE = 13\backslash)$, $\backslash(EA = 26\backslash)$, and $\backslash(\angle B = \angle E = 60^\circ\backslash)$, we need to find the

least possible value of $f(X) = AX + BX + CX + DX + EX$ for any point X in the plane. The answer is expressed as $m + n\sqrt{p}$, where m and n are positive integers and p is not divisible by the square of any prime. We need to find $m + n + p$.

1. **Coordinates Assignment**:
 - Place B at $(0, 0)$ and A at $(14, 0)$.
 - Calculate coordinates of C using $BC = 7$ and $\angle B = 60^\circ$:

$$C = \left(3.5, \frac{7\sqrt{3}}{2}\right)$$
 - Determine coordinates of D and E using the given side lengths and angles. Through complex calculations involving reflections and rotations, we find:

$$D = \left(\frac{313}{14}, \frac{169\sqrt{3}}{14}\right)$$

$$E = \left(\frac{72}{7}, \frac{104\sqrt{3}}{7}\right)$$
2. **Equilateral Triangles and Reflections**:
 - Construct equilateral triangles on AB and EA :
 - For AB , the third vertex F is $(7, 7\sqrt{3})$.
 - For EA , the third vertex G is $\left(\frac{241}{7}, \frac{65\sqrt{3}}{7}\right)$.
3. **Distance Calculation**:
 - Calculate the distance between the third vertices F and G :

$$FG = 16\sqrt{3}$$
4. **Sum of Distances**:
 - The minimal value of $f(X)$ is found by considering the sum of distances from a point that connects these equilateral triangles and the original vertices. The minimal sum is determined to be $40 + 16\sqrt{3}$.
5. **Final Answer**:
 - The values $m = 40$, $n = 16$, and $p = 3$ give the result $m + n + p = 40 + 16 + 3 = 59$.

$$\boxed{59}$$